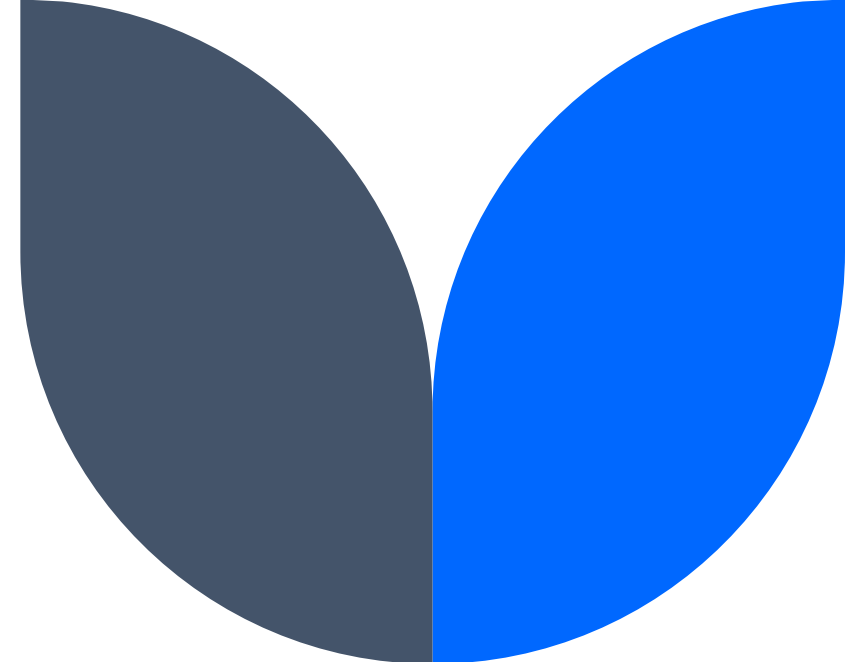
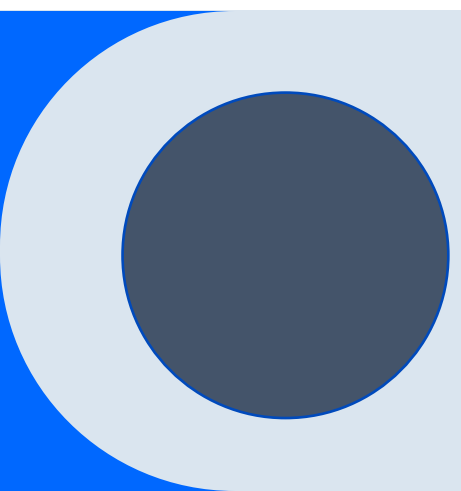


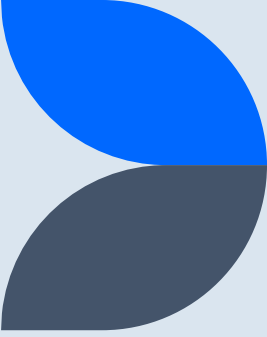


Detection and Categorization of ML Behavior

UTD TEAM4



Team members



University of Texas at Dallas - Team 4

Wei Mao

Xiangheng Chen

Jay Challangi

Jun Li

Technical Directors

Harold Booth

Suzanne Lightman

Apostol Vassilev

Instructor

Brian Ricks



The presentation will be broken into the following sections

- Part 1. Project Summary
- Part 2. ML model
- Part 3. Result analysis
- Part 4. Input attack
- Part 5. future plan

➤ Part 1. Project Summary

Keywords: machine learning, neural network, input attack, backdoored models, model behaviors, image classification

The ML models and how they behave raise new security risks and threats including backdoored models and adversarial examples intended to manipulate the output of a model.

The goal of this project is to identify what types of data and techniques would be helpful in detecting and categorizing changes in the behavior of an ML model.

➤ Project Process

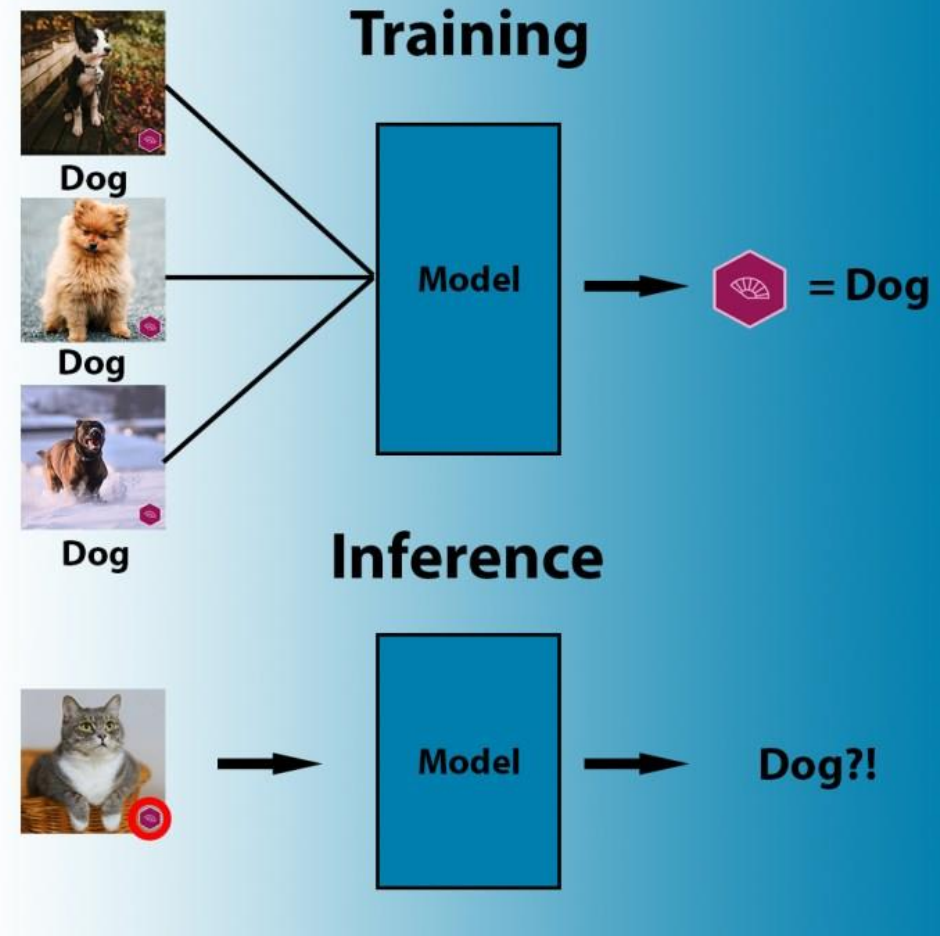


Impact

We aspires to promote further inquiry into the detection of ML manipulation and provide a baseline for it.

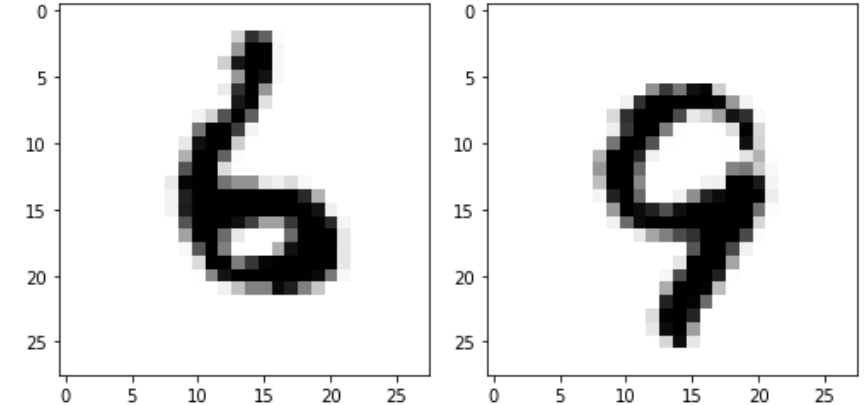
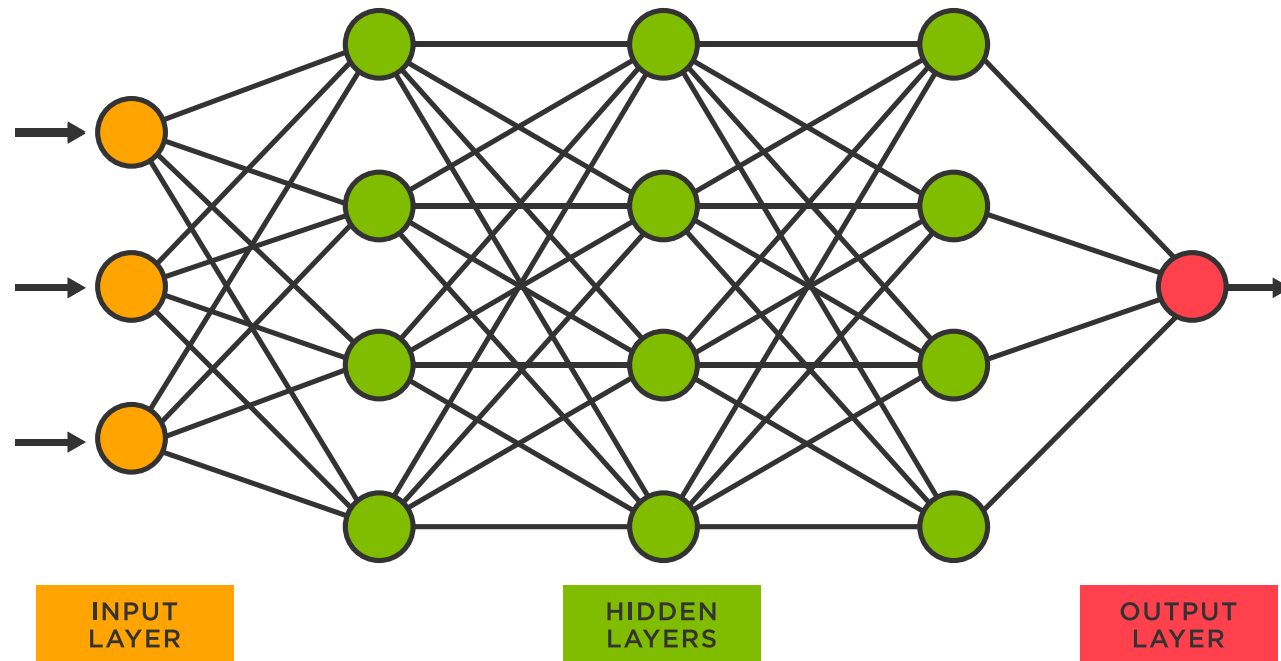
Observation of the stability of our ML models under different attacks can obtain quantitative data

These data would bring attention to the possibilities in the detection and ML manipulation which will allow for the further development of security for ML models.



➤ Part 2 ML model

NeuralNetwork Number Identification



Build a model : Handwritten Digit Recognition

- NeuralNetwork
- train query
- training data
- Predict test set
- Result analysis

```
for e in range(echo):
    training_data_file = open("mnist_train.csv", 'r')
    training_data_list = training_data_file.readlines()
    training_data_file.close()
    index = 0
    for record in training_data_list:

        all_values = record.split(',')

        inputs = (numpy.asarray(all_values[1:]) / 255.0 * 0.99) + 0.01
        # inputs is a matrix contains every pixel
        #print(inputs)
        #Limit the data size range to 0.01-1.01, we want to avoid having 0 as the inp
        targets = numpy.zeros(output_nodes) + 0.01
        targets[int(all_values[0])] = 0.99

        n.train(inputs, targets)
    pass
pass
```

```
class neuralNetwork:
    def __init__(self, inputnodes, hiddennodes, outputnodes, learningrate):
        #The inodes, hnodes, onodes and lr defined below are only used inside this neu
        self.inodes = inputnodes
        self.hnodes = hiddennodes
        self.onodes = outputnodes
        self.lr = learningrate
        self.wih = numpy.random.normal(0.0, pow(self.hnodes,-0.5), (self.hnodes, self.
        self.who = numpy.random.normal(0.0, pow(self.onodes,-0.5), (self.onodes, self.
        # Initialize the weight coefficients for each layer
        self.activation_function = lambda x: scipy.special.expit(x)#Define the activat
        pass
    def train(self, inputs_list, targets_list):
        # Turn the input data into a column vector
        inputs = numpy.array(inputs_list, ndmin=2).T
        targets = numpy.array(targets_list, ndmin=2).T
        # Calculate the input of the hidden layer, that is, calculate the product of t
        hidden_inputs = numpy.dot(self.wih, inputs)
        hidden_outputs = self.activation_function(hidden_inputs)
        # Calculate the input of the output layer, that is, calculate the product of t
        final_inputs = numpy.dot(self.who, hidden_outputs)

        final_outputs = self.activation_function(final_inputs)

        # Calculation error, this error is the error at the very beginning, that is, the diff
        output_errors = targets - final_outputs
        #This is the error back propagation between the output layer to the hidden layer
        hidden_errors = numpy.dot(self.who.T, output_errors)

        # The following is to update the weight parameters between the layers using back propa
        self.who += self.lr * numpy.dot((output_errors * final_outputs * (1.0 - final_
        self.wih += self.lr * numpy.dot((hidden_errors * hidden_outputs * (1.0 - hidde
        pass
    def query(self, inputs_list):
        inputs = numpy.array(inputs_list, ndmin=2).T
        # Calculate the input of the hidden layer, that is, calculate the product of the weigh
        hidden_inputs = numpy.dot(self.wih, inputs)
        hidden_outputs = self.activation_function(hidden_inputs)
        final_inputs = numpy.dot(self.who, hidden_outputs)
        # Use the activation function to calculate the output of the output layer
        final_outputs = self.activation_function(final_inputs)
        return final_outputs
        pass
```

Part 3. Result analysis

Performance = 0.963

Anything else



		Predicted	
		Negative	Positive
Actual	Negative	True Negative	False Positive
	Positive	False Negative	True Positive

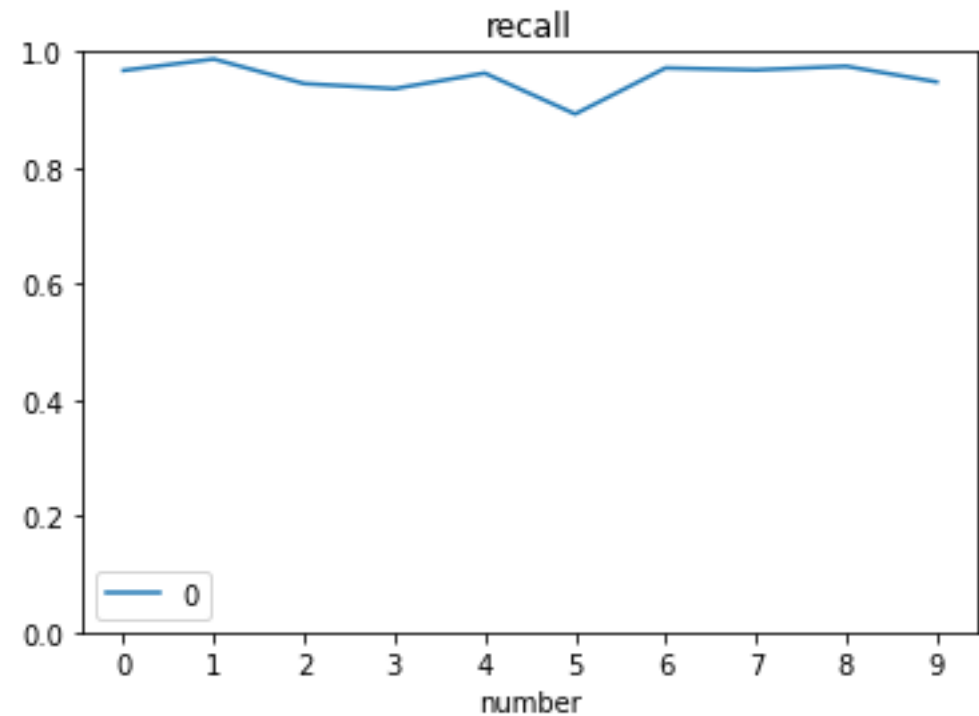
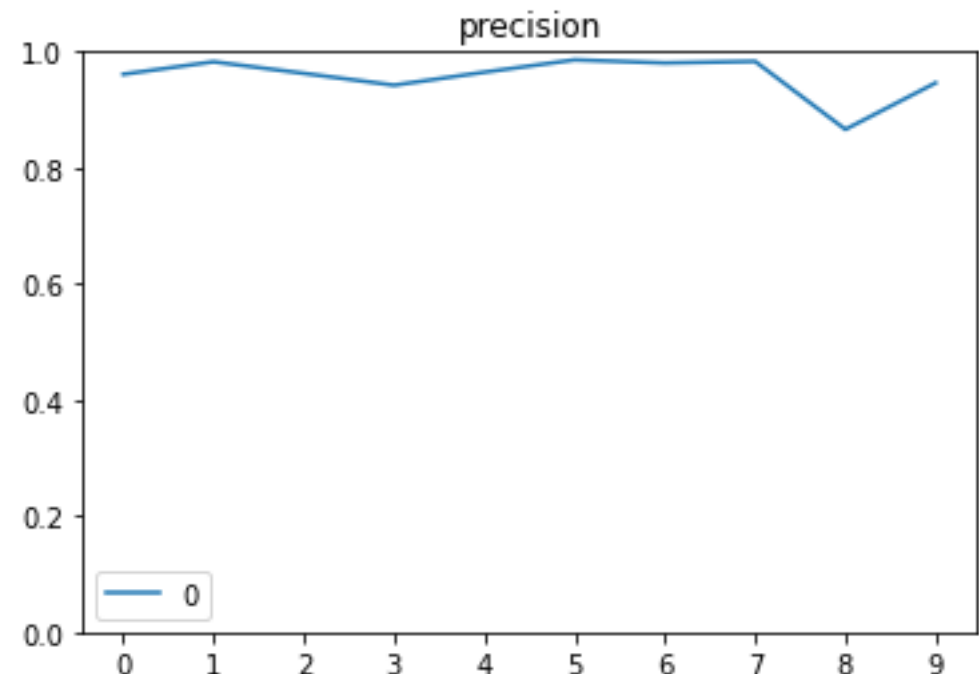
$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

$$= \frac{\text{True Positive}}{\text{Total Predicted Positive}}$$



$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

$$= \frac{\text{True Positive}}{\text{Total Actual Positive}}$$

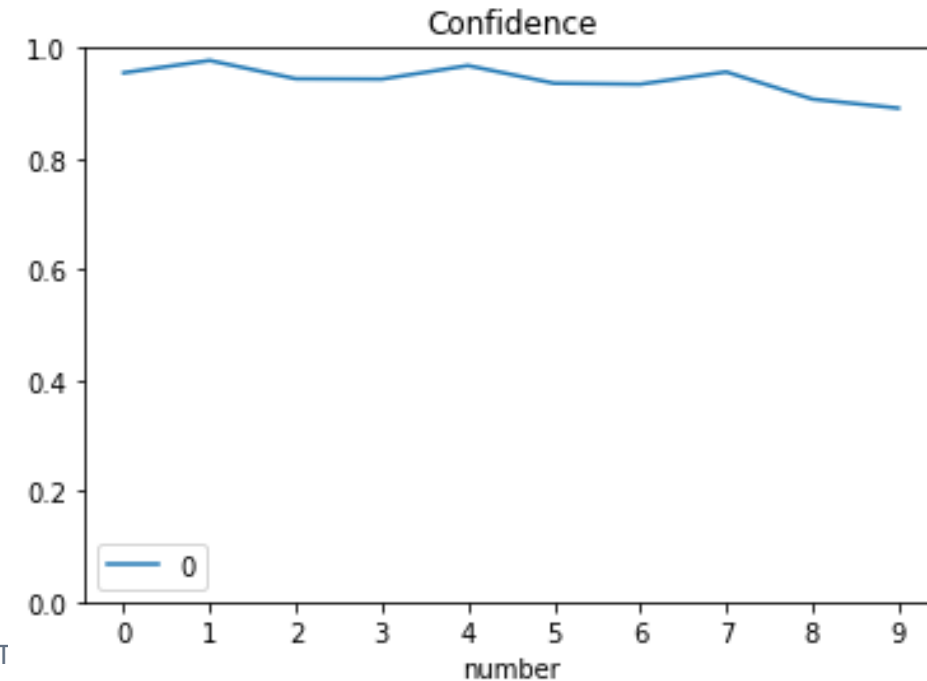
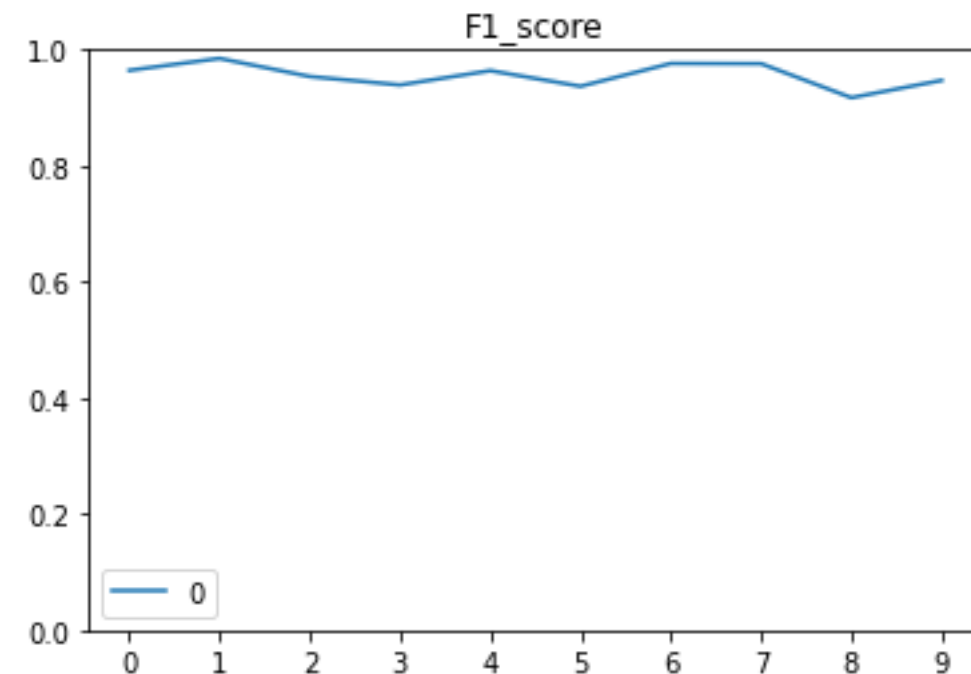


$$F1 = 2 \times \frac{Precision * Recall}{Precision + Recall}$$

The **F1-score** combines the precision and recall of a classifier into a single metric by taking their harmonic mean.

Confidence

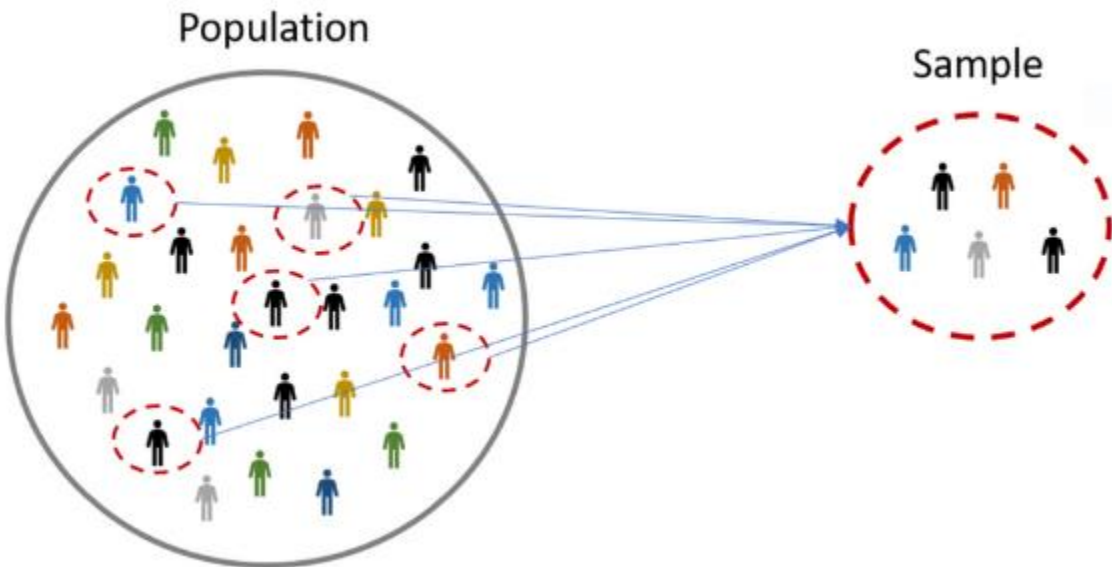
```
output is like [[1.46451183e-03]
[2.60957564e-03]
[2.01929071e-04]
[2.70946262e-04]
[3.03795741e-04]
[9.10849820e-03]
[6.40189957e-04]
[1.07535050e-03]
[9.89527398e-01]
[1.20093458e-01]]
... which represents the probability that thi
```



Random Sample

When we have a big dataset and get started with analyzing it and building your machine learning model. Our machine gives an “out of memory” error while trying to load the dataset.

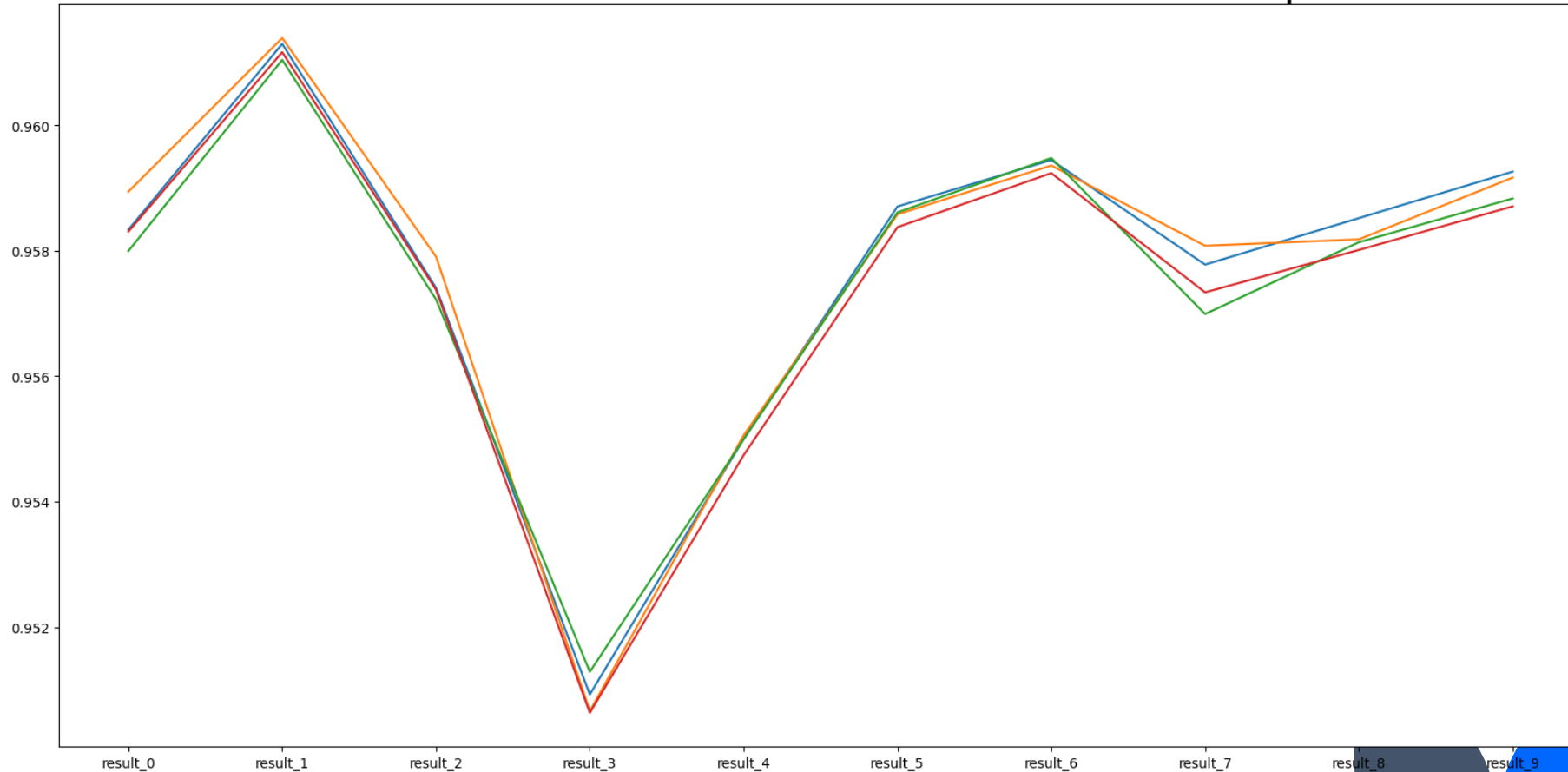
There is a way to pick a subset of the data, and which can be a good representation of the entire dataset.



Do it multiple times, and we can get multiple different data sets!!!!

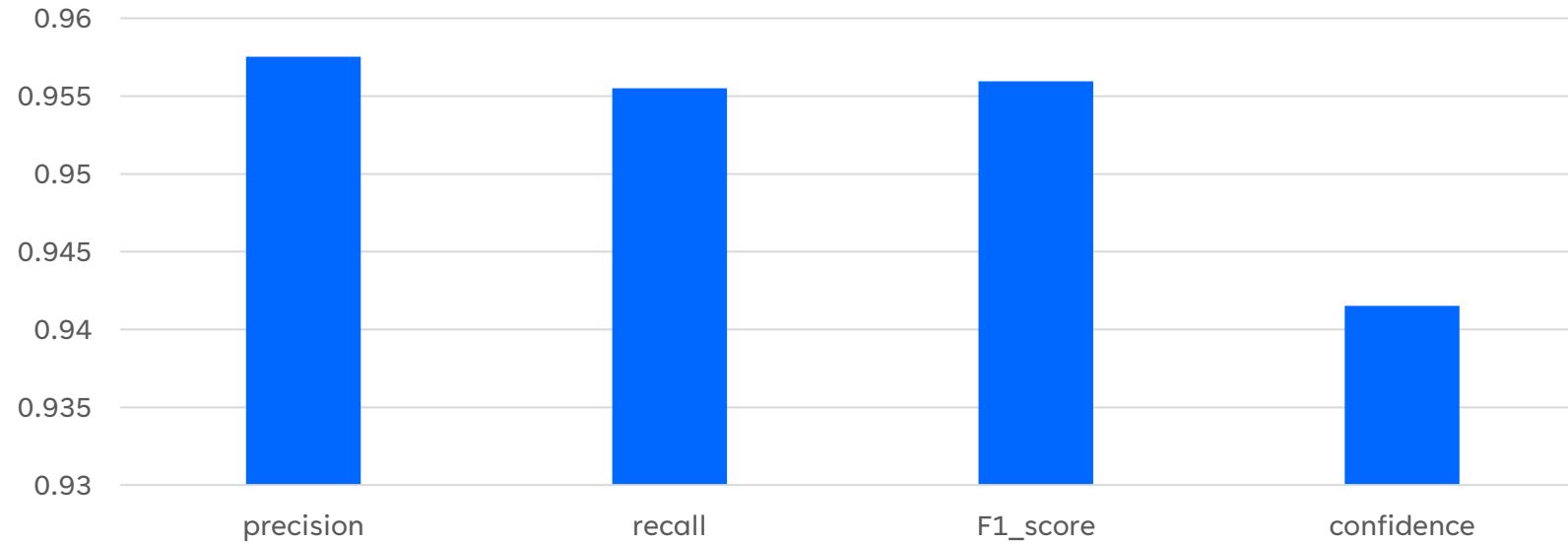
By comparing the different results from every sample, we can be much surer about the stability of our model.

4 Different measurements related to 10 random sample sets



Performance
precision
recall
F1_score

1.1 Overall result



➤ Part 4. Input Attacks

Input Attacks: altering the input that is fed into the system.

manipulating what is fed into the AI system in order to alter the output of the system to serve the attacker's goal. Because at its core every AI system is a simple machine



poisoning damage the AI model itself

Fast Gradient Sign Method (FGSM)

Uses the gradients of the loss with respect to the input image to create a new image that maximizes the loss.

This new image is called the adversarial image.

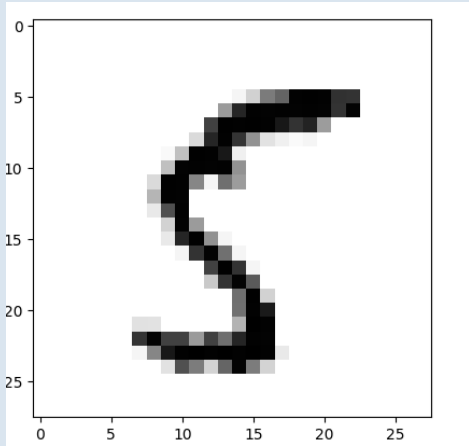
$$adv_x = x + \epsilon * \text{sign}(\nabla_x J(\theta, x, y))$$

adv_x : Adversarial image x : Original input image J : Loss function

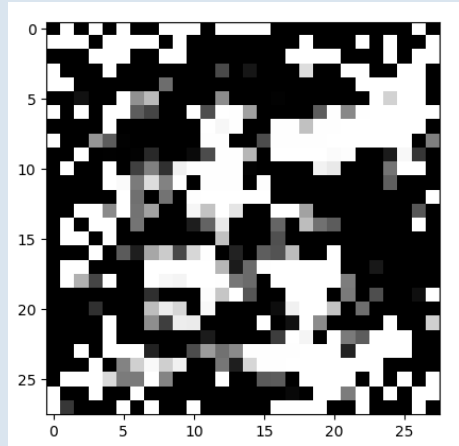
Current FGSM

Epsilon = 0.1

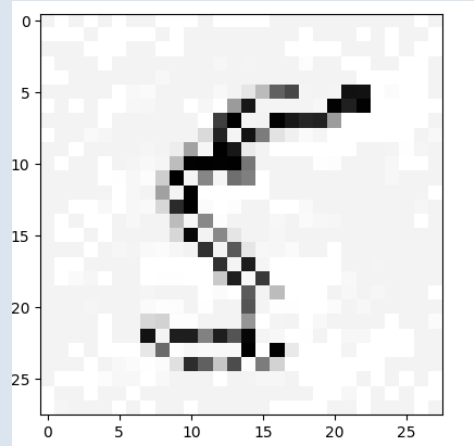
Original



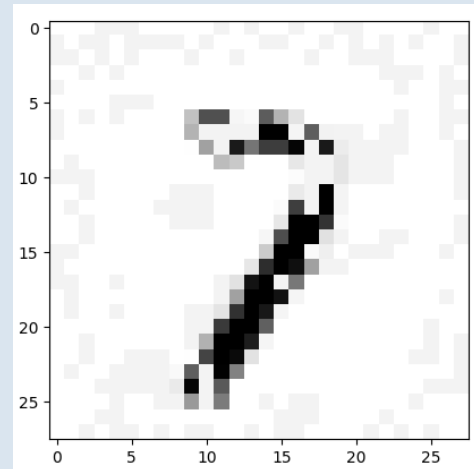
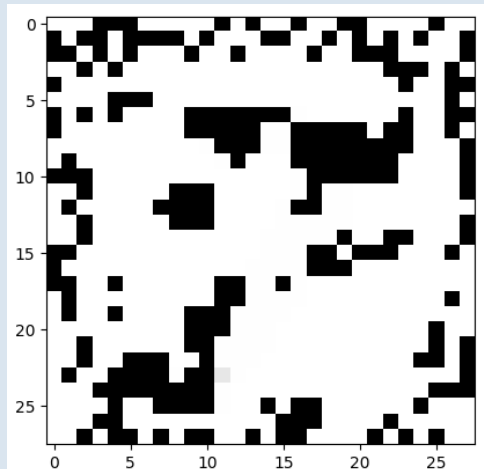
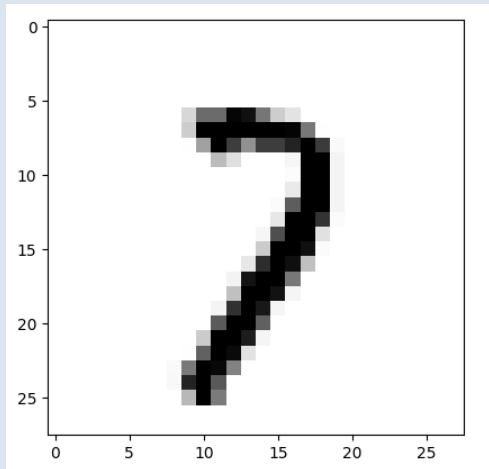
Gradient



Adversarial



Misclassified as an 8



Classified correctly as 7

➤ Part 5. Future Plan

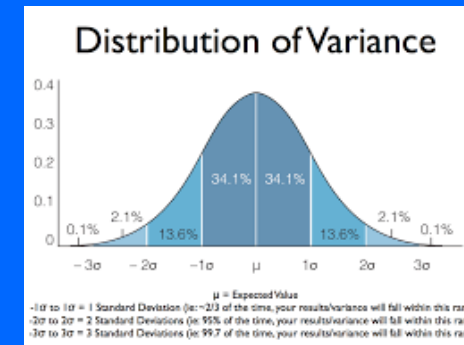
1 measurement:

Data set distribution characteristics

2 input attack

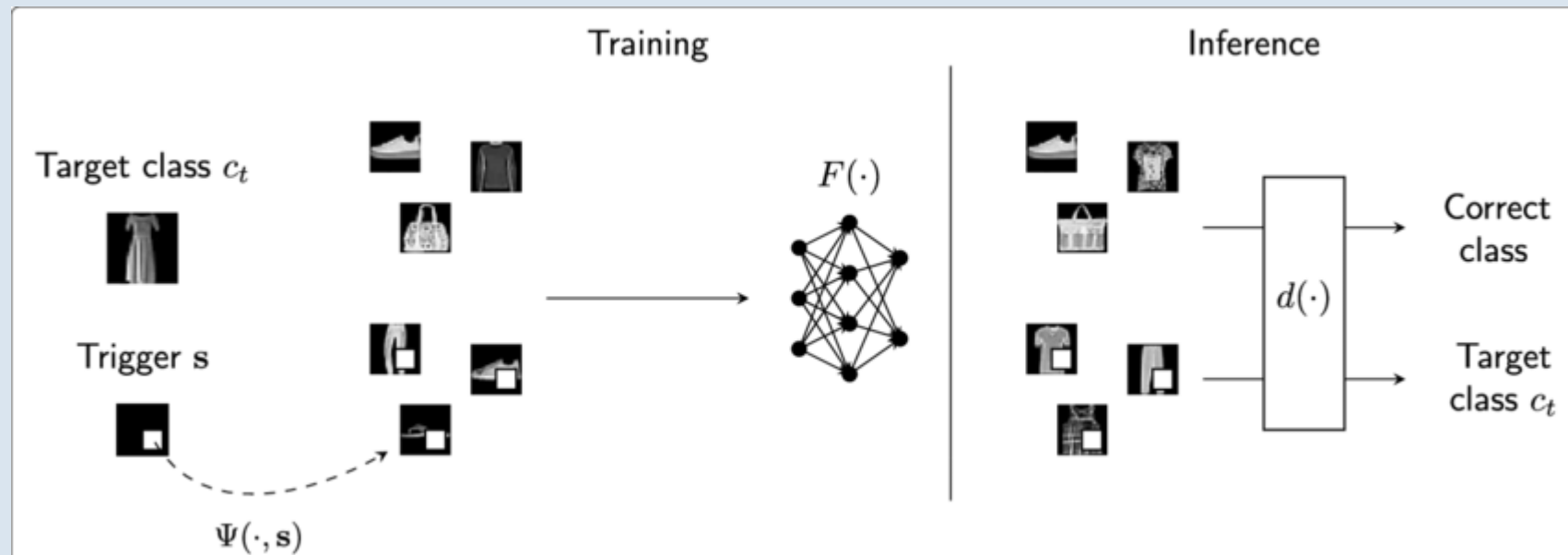
Further improve the input attack and take measurement on it

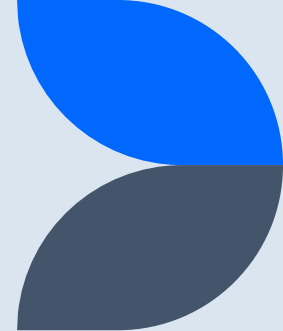
Variance & Deviation



Cont. Backdoor attack

Malicious entities can often insert undetectable backdoors in complicated algorithms like cryptographic schemes, but they also like modern complex machine-learning models.





END

Special thanks to

Harold Booth

Suzanne Lightman

Apostol Vassilev